Phage SECURITY

Robin Markets

Security Review

Conducted by:

Pyro, Lead Security Researcher
thekmj, Lead Security Researcher
YanecaB, Security Researcher

09.02.2026

Table of Contents

About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to Pyro.

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Robin Markets is a singleton yield bearing vault for Polymarket prediction market tokens. Users deposit YES/NO outcome token pairs, which the vault merges into USDC via Polymarket's Conditional Tokens Framework and deposits to external ERC4626 yield vaults to generate returns. Yield and losses are distributed back to depositors proportionally using TWAP snapshots, with all positions tracked through a multi layered accounting system and represented as composable ERC-1155 share tokens.

Executive summary

Overview

Project Name	Robin Markets
Repository	private
Commit hash	9f7605f7eb2776fb1f78c179dfcb9a61767f4a7a
Remediation	cf80c4f1625d7b3412e809ababa8cc79eb2dfb17
Public repo	d3b56a52ce91ab54b32b324714a0584daa211337
Methods	Manual review

Timeline

Audit kick-off	19.01.2026
End of audit	27.01.2026
Remediations start	22.02.2026
Remediations end	26.02.2026

Scope

src/RobinTimeLockController.sol

src/RobinStakingVault.sol

src/types/DataTypes.sol

src/mixins/AccountingMixin.sol

src/mixins/PausableMixin.sol

src/mixins/PolymarketMixin.sol

src/mixins/SignaturesMixin.sol

src/mixins/YieldStrategyMixin.sol

src/libraries/ShareMath.sol

src/libraries/TwapMath.sol

Issues Found

Severity	Count
High	4
Medium	2
Low	4

Findings Summary

ID	Title	Severity	Status
[H-01]	‘yieldIndex‘ accounts PnL wrong and will cause insolvency if yield goes negative	High	Fixed
[H-02]	Market can be prematurely finalized on non-TWAP required markets	High	Fixed
[H-03]	Two TWAPs will block each other and cause reverts most of the time through race condition	High	Fixed
[H-04]	Duplicate ID in batch transfers will break the accounting	High	Fixed
[M-01]	On TWAP-not-required markets, final TWAP can be dosed by front-running with any vault activity	Medium	Fixed
[M-02]	Some 4626 vaults may distribute rewards	Medium	Acknowledged
[L-01]	‘_weightedAverageSnapshot()‘ rounds down, causing dust overpayment of yield	Low	Fixed
[L-02]	Withdraws account yes/no amounts as if they are USDC	Low	Fixed
[L-03]	Insufficient isolation in the case a vault is compromised	Low	Fixed
[L-04]	‘setPauseAll‘ does not pause transfers	Low	Fixed

Findings

High Severity

[H-01] `yieldIndex` accounts PnL wrong and will cause insolvency if yield goes negative

`_calculateYieldIndexesWithPoolAssets` splits profits and losses to `yieldIndexYes` and `yieldIndexNo`. The flow consists of:

1. Getting the PnL in USDC, saved as `delta`

```
vars.isGain = marketValue > vars.principalContributed; // Detect
   potential losses since last time
vars.delta = vars.isGain
   ? marketValue - vars.principalContributed
   : vars.principalContributed - marketValue;
```

2. Getting the current variables in order to later calculate the needed split in `splitYieldWeighted`:

```
// totalSharesYes * yieldIndexYes / e118
vars.yesBaseline = ShareMath.sharesToAssetsWithIndex(
   vars.totalSharesYes, yieldIndexYes, false);

// totalSharesNo * yieldIndexNo / e118
vars.noBaseline = ShareMath.sharesToAssetsWithIndex(vars.totalSharesNo,
   yieldIndexNo, false);

vars.twapAccumulatorYesDelta = market.twapAccumulatorYes -
   market.lastYieldTwapCheckpointYes;
vars.timeDelta = market.lastTwapUpdate - market.lastYieldTimestamp;

(vars.yesDelta, vars.noDelta) =
   TwapMath.splitYieldWeighted(vars.delta,
      vars.twapAccumulatorYesDelta, vars.timeDelta, vars.yesBaseline,
      vars.noBaseline);
```

3. Splitting it proportionately to YES/NO shares, based on their weight:

```
// totalYield * yesWeightedValue / totalWeightedValue
yesYield = totalYield.mulDiv(yesWeightedValue, totalWeightedValue,
   Math.Rounding.Floor);
noYield = totalYield - yesYield; // Ensure no dust
```

4. And finally assigning the delta to `yieldIndexYes` as profit or loss:

```
    if (vars.totalSharesYes > 0 && vars.yesDelta > 0) {
       // yesDelta * 1e18 / totalSharesYes
       vars.yesChange = vars.yesDelta.mulDiv(DataTypes.INDEX_SCALE,
          vars.totalSharesYes, Math.Rounding.Floor);
       if (vars.isGain) {
          yieldIndexYes += vars.yesChange;
       } else {
          yieldIndexYes = yieldIndexYes > vars.yesChange
             ? yieldIndexYes - vars.yesChange
             : 1;
       }
    }
```

However in step 1 our delta is in USDC, but at step 3 and 4 we split it based on YES/NO weights, without taking into account that 1 USDC = 1 YES + 1 NO tokens. This means that when splitting PnLs to YES/NO yield indexes we count each YES/NO as 1 USDC.

This is best shown with the example in our POC:

https://gist.github.com/0x3b33/d6e46487fad4030818de7ca67cd10cb1

1. Alice deposits 1000 YES
2. Bob deposits 1000 NO
3. 200 USDC loss is applied
4. That loss is splits between YES and NO for 10% decrease in both
5. `yieldIndexYes` and `yieldIndexNo` are set to 0.9

The vault thinks that we have 900 YES and 900 NO tokens inside, even though we only have only 800 USDC in the vaults.

```
// Result by calling `getMarketAssets`
Actual USDC in vault:  800000000
Computed yesAssets: 900000000
Computed noAssets:  900000000
```

This is also the same for increase in assets, the `yiledIndex` for both YES and NO tokens will increase 2x less than it should.

The only reason this does not cause an issue when yield is increased is because withdraws also double count USDC inconsistently, which is described in issue #13.

Recommendation

Redo the accounting distribution.

[H-02] Market can be prematurely finalized on non-TWAP required markets

Summary

On non-TWAP required markets, the validation for signed TWAP short-circuits early, allowing malicious TWAP data to pass. This allows prematurely finalizing markets, skewing all yields to one side.

Vulnerability details

The root cause is in the following validation:

```
if (vars.needsTwapSignatureVerification && !getGlobalTwapDisabled(
   ) && !_verifyBatchTwapSignature(twapData)) {
   revert InvalidTwapSignature();
}
```

Notice that if the TWAP is not required, either because the market doesn't enforce TWAP or the global disable flag is on, then the check **short-circuits** i.e. `_verifyBatchTwapSignature()` is never reached, and the signature validity is never checked

That means it is possible to submit any TWAP data with a positive `marketEndedAt`, and `_processTwapForMarket` will think the market is already finalized, and will apply the final TWAP right away, without a proper signature, regardless of timing.

Impact

All yields can be skewed to one side (possibly the wrong side) before the market truly finalizes. Yield can be stolen.

Recommendation

Always validate the signed TWAP data if it's non-empty.

[H-03] Two TWAPs will block each other and cause reverts most of the time through race condition

Summary

Overly strict validation on `validateTwapTiming` will cause race conditions in practice.

Vulnerability details

For TWAP-required markets, all deposits and withdrawals must be accompanied with a TWAP data. There are certain requirements for a TWAP data to be considered valid:

```
if (startTimestamp != lastUpdate) return false;
if (endTimestamp + gracePeriod < currentTime) return false;
```

Suppose there are two users interacting at the same time (about a minute apart). Both users need to get a signed TWAP data, taken from the API.Then the second user is guaranteed to revert on the first condition, because both users' data have the same `startTimestamp`, but the first user's TWAP would have gotten through first, blocking the second user's tx entirely.

This can be weaponized to block all vault activity as so (up until market maturity): - Every minute, request a TWAP from the backend, but do not submit - Whenever a user does a vault activity, front-run them with any vault activity (minimal deposit or withdraw)

Impact

Two TWAPs will block each other and cause reverts most of the time through race condition. This makes it weaponizable to lock user funds.

Recommendation

One possible solution is that, if the condition `startTimestamp > lastUpdate` holds, update only from `lastUpdate` up to `endTimestamp` with the given price.

[H-04] Duplicate ID in batch transfers will break the accounting

The `_update` function in `AccountingMixin` overrides the standard ERC-1155 hook to handle the proportional transfer of principal (`tokensYes`/`tokensNo`) alongside share transfers. The function calls `super._update` (which updates the ERC-1155 balances) before executing the custom accounting logic in `_handleTransferAccounting`.

```
function _update(address from, address to, uint256[] memory ids,
   uint256[] memory values) internal virtual override {
...
super._update(from, to, ids, values);
     if (isTransfer) {
        for (uint256 i = 0; i < ids.length; i++) {
           if (values[i] > 0) {
              _handleTransferAccounting(from, to, ids[i], values[i]);
           }
        }
```

Inside `_handleTransferAccounting`, the contract attempts to reconstruct the sender's pre-transfer balance to calculate the correct principal-to-share ratio:

```
uint256 senderSharesBefore = balanceOf(from, tokenId) + sharesTx;
```

This logic fails if a batch transfer contains duplicate token IDs (e.g., `ids=[idA, idA]`, `values=[10, 10]`).

1. `super._update` decrements the sender's balance by the total amount for that ID (e.g., -20) immediately.
2. When the loop processes the first occurrence of the ID, `balanceOf` returns the fully decremented balance.
3. The reconstruction adds back only the current index's amount (`+10`), resulting in a `senderSharesBefore` that is lower than reality (e.g., calculated as 90 instead of 100).
4. This artificially inflates the transfer ratio (`sharesTx / senderSharesBefore`), causing the sender to transfer significantly more principal (`tokensYes`) than they should.

Leading to various inaccuracies in internal accounting and, eventually, insolvency during withdrawals.

Recommendation

Disallow batch transfers that contain duplicate `tokenId` values by enforcing that the `ids` array is strictly sorted and contains no repetitions.

Medium Severity

[M-01] On TWAP-not-required markets, final TWAP can be dosed by front-running with any vault activity

Summary

On TWAP-not-required markets, due to the TWAP being permissionlessly updated by default, the final TWAP can be dosed by front-running with any activity, overriding the updated time.

This makes it possible to deny updating the final TWAP (that resolves the market).

Vulnerability details

There are two factors contributing that makes it possible: - On non-TWAP required markets, TWAP data is not required, and any activity updates the TWAP with the default 50/50 price. - The `validateTwapTiming` forces any TWAP updates to match the last update timestamp

```
if (startTimestamp != lastUpdate) return false;
```

`TwapData.startTimestamp` is part of the signed TWAP data by the backend, but `lastUpdate` can be permissionlessly triggered. Hence it is possible to force-fail the validation by simply updating the TWAP with any activity

Consider the following scenario:

- The backend signs a new Twap data for the time interval $[1000; 1200)$. By `validateTwapTiming`, the `lastUpdate` timestamp must be **exactly** 1000.
- Now, if the market doesn't require a Twap, then default TWAP applies. Any activity done on the vault will push `lastUpdate` up to the current timestamp (which is at least 1200), thereby invalidating any existing Twap signatures.

The result is that the Market cannot be finalized, and post-finalization yield distribution will be inaccurate.

Impact

Post-finalization yield distribution will be inaccurate. Holders of the losing side can prevent the final TWAP from being applied, which will allow siphoning yield from the winning side.

Recommendation

The logic for final TWAP on non-required markets should require only the market finalization timestamp, the pre-finalization price, and the finalization price.

[M-02] Some 4626 vaults may distribute rewards

Some 4626 vaults (like Morpho, Yarn, Euler) distribute rewards to depositors on a somewhat consistent basis.

However in the current code there is no way to claim or handle them, meaning they will be stuck inside `RobinStakingVault.sol`.

Recommendation

Consider adding either a call function accessible by the owner to claim the rewards, swap them to USDC and distribute them.

Another alternative (that will require a lot of code) is to implement custom modules for each vault type. This way you would be able to more easily integrate with all kinds of vaults (even ones that are not 100% 4626 compliant).

Low Severity

[L-01] `_weightedAverageSnapshot()` rounds down, causing dust overpayment of yield

All usages of `_weightedAverageSnapshot()` are to compute the new user's yield index after they've received some shares, by taking the weighted average of the two share portions (already owned versus received).

However this function rounds down, meaning the user's yield index is slightly lower than it is, causing the vault to overpay the dust portion when those shares are withdrawn.

Recommendation

`_weightedAverageSnapshot()` should round up, so that the yield index favors the vault rather than the user receiving the shares.

[L-02] Withdraws account yes/no amounts as if they are USDC

This issue relates to the previous High-severity finding regarding the YES/NO `yieldIndex`.

During withdrawal, the protocol calculates the total assets associated with the shares and then determines the yield to be distributed in USDC:

```
// Check for loss
bool yesLoss = yesAmount > result.yesAssets;
bool noLoss = noAmount > result.noAssets;

// Get amounts without yield
result.actualYesAmount = yesLoss
   ? result.yesAssets
   : yesAmount;

result.actualNoAmount = noLoss
   ? result.noAssets
   : noAmount;
```

Next, the protocol calculates the shortfall of YES/NO tokens to determine how much USDC is required for splitting:

```
    result.splitNeeded = yesShortfall > noShortfall
       ? yesShortfall
       : noShortfall;
```

Notice that `result.splitNeeded` is denominated in USDC. The code adds the yield from both YES and NO tokens to this value as if each token were worth 1 USDC individually. In reality, while the yield is paid in USDC, a YES token and a NO token only equal 1 USDC when combined. By summing them separately, the calculation incorrectly treats them as two independent USDC amounts rather than a single collateralized pair.

```
    result.yieldNeeded =
       (result.yesAssets > result.actualYesAmount
          ? result.yesAssets - result.actualYesAmount
          : 0)
       + (result.noAssets > result.actualNoAmount
          ? result.noAssets - result.actualNoAmount
          : 0);

    result.totalNeeded = result.splitNeeded + result.yieldNeeded;
```

Recommendation

If the previous High-severity finding is resolved by changing how the YES/NO `yieldIndex` functions, this logic must be updated accordingly to ensure USDC yield is not double-counted.

[L-03] Insufficient isolation in the case a vault is compromised

Supposing one of the underlying vaults is compromised for whatever reason. There are several issues `YieldStrategyMixin` that makes it impossible to truly exit a vault if it's ever compromised:

- As soon as a vault is added, infinite USDC approval is given to the vault.
- The infinite approval is removed only when the vault is removed. However `_withdrawAllFromVault()` will fail if the full withdrawal fails.

```
function _removeVault(address vault) internal returns (uint256 withdrawn) {
   // ...
   withdrawn = _withdrawAllFromVault(vault); // @audit this reverts if not all
      shares can be withdrawn

   IERC20($.underlyingUsdc).forceApprove(vault, 0);
```

That means it might not be possible to remove the hanging approval entirely.

Then even if the vault has its emergency mode activated, and as many funds as possible are withdrawn, there is still a hanging infinite approval to the vault, and the funds are still movable by the vault if for any reason.

Recommendation

In general, it is not a good practice to leave hanging ERC-20 approvals. It is recommended that, whenever a deposit to vaults is needed, explicitly approve the amount to spent, and possibly reset the allowance to zero afterwards.

[L-04] `setPauseAll` does not pause transfers

`setPauseAll` function exists, but it only pauses deposits and withdraws, as currently `_update` is paused only by `$.transfersPaused`:

```
function _update(address from, address to, uint256[] memory ids,
   uint256[] memory values) internal virtual override {
   AccountingStorage storage $ = _getAccountingStorage();

   // Check transfer pause (only for actual transfers, not mints/burns)
   bool isTransfer = from != address(0) && to != address(0);
   if (isTransfer && $.transfersPaused) {
      revert TransfersPaused();
   }

   // ...
}
```

Recommendation

Make sure `_update` also gets paused when `setPauseAll` is triggered.